12

Лекция 27–28 апреля

Разрешимость и теорема Райса

Константин Чухарев

Дискретная математика — Весна 2027

Три класса языков

> *Мы должны знать. Мы будем знать.*
>
> — ***Давид Гильберт***

Разрешитель отвечает на каждом входе

Хочется инструмент, который по описанию программы и её входу заранее говорит, завершится ли она.

Определение 1 (Разрешимость (*decidability*))

Язык L **разрешим**, если существует машина Тьюринга D, которая на любом входе w останавливается и принимает его тогда и только тогда, когда $w \in L$.

Отрицательный ответ — тоже остановка, поэтому зацикливаться разрешителю нельзя.

Пример (Разрешимый язык)

Язык $a^n b^n$ разрешим — машина вычёркивает крайние непомеченные пары ab.

Вход aabb идёт путём aabb $\to$ XabX $\to$ XXXX и принимается, а aab оставляет XaX и отвергается, когда второму проходу не хватает b.

Язык без разрешителя тоже существует — HALT распознаваем, но неразрешим.

Распознаватель даёт гарантию только для своих

Симуляция надёжна только на «да», а зацикливание не наблюдается.

Определение 2 (Распознаваемость (*recursively enumerable*))

Язык L **распознаваем**, если существует машина Тьюринга, которая принимает в точности слова L. Каждый вход из L она принимает, а каждый вход вне L не принимает.

На входах вне L распознаватель может зациклиться — определение не обязывает его останавливаться.

Пример («HALT» распознаваем)

U симулирует M на w и принимает при остановке симуляции — на чужих словах симулятор крутится вечно.

Ко-распознаватель даёт гарантию для чужих

Прогон на миллион шагов и прогон без остановки неразличимы на любом конечном шаге, поэтому зацикливание от медленности не отличить. У симуляции есть зеркальный двойник — машина, отвечающая за слова вне языка.

Определение 3 (Ко-распознаваемость (*co-recognizability*))

Язык L **ко-распознаваем**, если распознаваем его дополнение $\overline{L}$.

Распознаватель дополнения принимает каждый «нет»-вход и может зациклиться на каждом «да»-входе.

Пример (Дополнение «HALT»)

$\overline{\mathrm{HALT}}$ — пары $(\langle M\rangle, w)$, на которых M зацикливается.

«Да»-свидетель здесь — бесконечность пробега, а бесконечность не предъявить за конечное время.

Распознаваемость и ко-распознаваемость

Два свойства независимы, и у языка может оказаться только одна из двух сторон.

Язык	Распознаваемость	Ко-распознаваемость
$a^n b^n$	есть	есть
HALT	есть	нет
$\overline{\text{HALT}}$	нет	есть

Разрешимый язык несёт обе стороны, и инверсия ответа разрешителя даёт готовый ко-распознаватель.

Дополнение не наследуется
Симуляция даёт конечный принимающий прогон, бесконечный пробег не предъявить — у HALT и его дополнения ровно по одной стороне. Распознаваемость не наследуется дополнением, потому что про чужие слова распознаватель ничего не знает.

Перечислитель языка

Определение 4 (Перечислимость (*enumerability*))

Язык L **перечислим**, если есть МТ E — **перечислитель** — печатающая все слова L, каждое хотя бы раз и ничего кроме.

Одна сила печатает, другая проверяет — теорема уравнивает.

Пример (Перечислитель для «HALT»)

На этапе s машина E делает по s шагов каждой симуляции из первых s машин $M_1, ..., M_s$ на первых s входах $w_1, ..., w_s$.

Остановилась симуляция пары $(\langle M_i \rangle, w_j)$ — E печатает $\langle M_i \rangle w_j$ и к паре больше не возвращается. Пара (i, j) дожидается своего этапа $s \geq \max(i, j)$, поэтому каждая останавливающаяся пара напечатана.

Зацикливающиеся пары не печатаются никогда — перечислен в точности HALT.

Перечислимость и распознаваемость

Теорема связывает две интуиции — печать и проверку.

Теорема 1 (Перечислитель равносилен распознавателю)

L перечислим тогда и только тогда, когда L распознаваем.

Доказательство

Докажем эквивалентность в обе стороны.

Пусть сначала E — перечислитель L. Распознаватель запускает E и принимает, как только w появился на печати. Чужие слова не появятся никогда.

Обратно, чередуем шаги распознавателя M по всем входам и печатаем вход в момент принятия.

Каждое слово из L будет принято и напечатано, слова вне L — нет.

Разрешимость с двух сторон

Ко-распознаватель — вторая половина разрешителя, и две половины складываются в целое.

Теорема 2 (Разрешимость равносильна распознаваемости с двух сторон)

L разрешим тогда и только тогда, когда распознаваемы L и $\overline{L}$.

Доказательство

Докажем эквивалентность в обе стороны. Пусть L разрешим. Тогда разрешитель D служит распознавателем L, а инверсия его ответа — распознавателем $\overline{L}$.

Обратно, распознаватели M и N запускаем параллельно, чередуя шаги. Ровно один язык содержит вход, и его приём даёт верный ответ.

Для HALT вторая половина отсутствует — ниже это станет теоремой.

Замкнутость разрешимых

Класс ценен тогда, когда операции не выводят из него.

Теорема 3 (Замкнутость разрешимых языков)

Разрешимые языки замкнуты относительно объединения, пересечения и дополнения.

Доказательство

Докажем построением, операция за операцией.

Объединение. Запускаем D_1 и D_2 на одном входе по очереди и принимаем, если приняла хотя бы одна. Обе останавливаются, поэтому останавливается и составная.

Пересечение. Так же, только принимаем после двух приёмов подряд.

Дополнение. Меняем ответ D на противоположный.

Три операции над простыми числами

Разрешимость булевых операций не требует новых алгоритмов, только соединения готовых.

Пример (Объединение, пересечение и дополнение)

Возьмём язык простых чисел и язык чётных чисел. Перебор делителей до $\sqrt{n}$ разрешает первый — для 91 он находит 7, для 97 упирается в 9 — а чётность читается по последней цифре.

Пересечение — это простые среди чётных, то есть одна двойка, и обе машины принимают её по очереди. Объединение принимает всё, что приняла хотя бы одна, а дополнение к разрешимому языку получается инверсией ответа.

Одна и та же схема годится для любой пары разрешимых языков, потому что соединение всегда останавливается.

Замкнутость распознаваемых

У распознавателей одна гарантия — приём своих слов, поэтому и замкнутость неполная.

Теорема 4 (Замкнутость распознаваемых языков)

Распознаваемые языки замкнуты относительно объединения и пересечения.

Замкнутость относительно дополнения у класса отсутствует — контрпримером служит проблема остановки.

Доказательство

Для объединения чередуем шаги M_1 и M_2 и ждём первого приёма — без чередования зацикливание первой машины не дало бы хода второй.

Для пересечения чередование не нужно. Слово из пересечения обе принимают за конечное время, а слово вне одной из частей застревает.

Проблема остановки

> *Машины удивляют меня с завидной регулярностью.*
>
> ***— Алан Тьюринг***

Универсальная машина и программа как данные

Диагональ опирается на умение машины исполнять другие машины по описанию.

Теорема 5 (Универсальная машина)

Существует машина Тьюринга U, которая по паре $(\langle M \rangle, w)$ симулирует работу M на входе w.

Доказательство

Докажем построением. Кодировка $\langle M \rangle$ — конечная строка с таблицей переходов.

U держит конфигурацию симулируемой машины на ленте и выполняет шаги по таблице — каждый шаг M рано или поздно воспроизводится.

Самоприменение законно, ведь $\langle M \rangle$ — такая же строка, как w.

Интерпретаторы и виртуальные машины — инженерная жизнь теоремы.

Постановка проблемы остановки

Теперь вопрос, который диагональ отправит в неразрешимые. Завершится ли программа на данном входе?

Определение 5 («HALT»)

По паре $(\langle M \rangle, w)$ нужно узнать, останавливается ли M на входе w.

$$\mathrm{HALT} = \{\langle M \rangle w \mid M \text{ останавливается на } w\}.$$

Пример (Два конкретных запроса)

Машина M_{loop} при любом символе остаётся в q_0, например $(q_0, a) \to (q_0, a)$. Поэтому $\langle M_{\text{loop}} \rangle 0 \notin \mathrm{HALT}$.

Машина M_{acc} переходит в состояние принятия при любом символе — тогда $\langle M_{\text{acc}} \rangle \varepsilon \in \mathrm{HALT}$.

Диагональная машина D

Допустим, существует разрешитель H. На входе $\langle M \rangle w$ она всегда останавливается и отвечает в точности по остановке M.

Строим машину D по трём правилам.

1. На входе x машина D запускает H на паре (x, x) и спрашивает, остановится ли машина с кодом x на собственном коде.
2. Если H приняла, машина D зацикливается.
3. Если H отвергла, машина D принимает.

Почему собственный код — не фокус?
Круга в построении нет. Машина D строится после H, а строка $\langle D \rangle$ существует, как код любой машины.

Вопрос «что сделает D на $\langle D \rangle$» задаётся после построения D. Анализатор подаёт машине её собственный текст, как компилятор читает сам себя.

Подстановка D в собственный код

Остался один ход — подставить D в самое себя.

Теорема 6 (Неразрешимость проблемы остановки)

Язык HALT неразрешим.

Доказательство

Докажем от противного. Пусть H решает HALT, D построена, рассмотрим $D(\langle D \rangle)$.

Если D остановилась, сработала ветка «отвергла» — H ответила «зациклится», но D остановилась.

Если D зациклилась, сработала ветка «приняла» — H ответила «остановится», но D зациклилась.

Оба случая делают H ошибочной — противоречие.

Распознаваемый, но не разрешимый

Диагональ запретила решатель с «да» и «нет», а симуляция даёт лишь «да».

Теорема 7 («HALT» вне разрешимых)

HALT распознаваем, но не разрешим, а его дополнение даже не распознаваемо.

Доказательство

Распознаваемость. Машина U принимает $\langle M \rangle w$ в момент остановки M и в противном случае крутится вместе с ней.

Нераспознаваемость дополнения. Пусть N распознаёт $\overline{\text{HALT}}$. На входе $\langle M \rangle w$ запустим U и N параллельно, чередуя шаги.

Ровно одна машина примет — U при остановке M, N при зацикливании.

Её приём — верный ответ на вопрос остановки — противоречие с диагональю.

Карта четырёх классов

Каждая неразрешимость уточняет карту, поэтому соберём классы и их замкнутости в одну таблицу.

Класс	Машина	Объединение	Пересечение	Дополнение
Регулярные	ДКА	да	да	да
Контекстно-свободные	МП-автомат	да	нет	нет
Разрешимые	разрешитель	да	да	да
Распознаваемые	распознаватель	да	да	нет

Клетка «нет» в последней строке — теорема о дополнении HALT. У языка есть распознаватель, а у дополнения его нет.

Сведения

> *Лучше решить правильную задачу неправильным способом, чем неправильную задачу правильным.*
>
> ***— Алан Перлис***

Сведение как перенос вопроса

Диагональ строят один раз — дальше её переиспользуют.

Определение 6 (Сведение (*many-one reduction*))

Язык A **сводится** к B, пишется $A \leq_m B$, если есть вычислимая тотальная функция f с условием

$$w \in A \leftrightarrow f(w) \in B.$$

Пример (От остановки к пустому входу)

Сведём остановку к $\mathrm{HALT}_\varepsilon$, языку машин с остановкой на пустом входе.
Отображение $f(\langle M \rangle, w) = \langle M' \rangle$, где M' записывает w и работает как M.

Если M остановилась на w, то M' тоже, поэтому $\langle M' \rangle \in \mathrm{HALT}_\varepsilon$. Если M зациклилась, то M' крутится после записи, поэтому $\langle M' \rangle \notin \mathrm{HALT}_\varepsilon$.

Логика переноса

Из определения сведения сразу видно, как переносятся алгоритмы и их отсутствие.

Теорема 8 (Что переносит сведение)

Пусть $A \leq_m B$.

- Если B разрешим, то разрешим и A.
- Если A неразрешим, то неразрешим и B.
- Если B распознаваем, то распознаваем и A.
- Если B ко-распознаваем, то ко-распознаваем и A.

Доказательство

Докажем построением. Разрешитель A строит образ $f(w)$ и отдаёт его разрешителю B, а условие $w \in A \leftrightarrow f(w) \in B$ делает ответ верным на каждом входе.

Неразрешимость — контрапозиция, остальные два пункта — та же подстановка f.

Куда идёт сведение

Теорема читается в одну сторону, и путаница в направлении — частая ошибка.

Пример (Стрелка из лёгкого в трудное)

Возьмём машину M, которая сразу принимает, и отображение $f(w) = \langle M \rangle w$.

Функция тотальна и вычислима, а образ лежит в HALT всегда, поэтому $\Sigma^* \leq_m \text{HALT}$. Левая часть разрешима, посылка контрапозиции не выполнена, и стрелка ничего не сообщает.

Ошибка направления

Рабочий ход — сведение уже неразрешимого языка к изучаемому. Из $A \leq_m B$ и неразрешимости A следует неразрешимость B, а сведение $B \leq_m A$ не говорит ничего.

Пустота языка

Первая остановка на маршруте сведений — вопрос о пустоте языка машины.

Пример (От HALT к E_{TM})
$E_{\mathrm{TM}} = \{\langle M \rangle \mid L(M) = \varnothing\}$.

Сводящая функция задаётся формулой $f(\langle M \rangle, w) = \langle M' \rangle$. Машина M' на любом входе игнорирует его, симулирует M на w и принимает при остановке симуляции.

Если M остановилась на w, то $L(M') = \Sigma^*$, а если зациклилась, то $L(M') = \varnothing$.

Разрешитель E_{TM} с инверсией ответа решал бы HALT — значит, $\mathrm{E_{TM}}$ неразрешим.

Проблема тотальности

Остановка спрашивает про один вход, а тотальность — сразу про все.

Определение 7 (Проблема тотальности)

По описанию машины M нужно узнать, останавливается ли она на каждом входе.

$$\mathrm{TOT} = \{\langle M\rangle \mid \forall w.\, M \text{ останавливается на } w\}.$$

Пример (Сведение от остановки)

По паре $(\langle M\rangle, w)$ строим M', которая на входе x игнорирует x, симулирует M на w и принимает.

Поведение M' от входа не зависит. Остановилась M на w — M' останавливается на каждом x, и $\langle M'\rangle \in \mathrm{TOT}$. Зациклилась — M' крутится на каждом x, и $\langle M'\rangle \notin \mathrm{TOT}$.

Разрешитель TOT решал бы HALT, значит, проблема тотальности неразрешима.

Эквивалентность программ

При рефакторинге первым делом спрашивают, вычисляют ли две версии кода одно и то же.

Теорема 9 (Неразрешимость эквивалентности)

Язык $\mathrm{EQ}_{\mathrm{TM}} = \{\langle M_1 \rangle \langle M_2 \rangle \mid L(M_1) = L(M_2)\}$ неразрешим.

Доказательство

Докажем сведением от пустоты языка к эквивалентности. Пусть $f(\langle M' \rangle) = \langle M' \rangle \langle M_{\text{loop}} \rangle$, где M_{loop} не принимает ничего.

Если язык M' пуст, то $L(M') = \varnothing = L\big(M_{\text{loop}}\big)$ и образ лежит в $\mathrm{EQ}_{\mathrm{TM}}$. Если язык M' непуст, то образ лежит вне $\mathrm{EQ}_{\mathrm{TM}}$.

Решатель эквивалентности решал бы тогда пустоту, а она неразрешима.

Определение соответствий Поста

Сведения работают и вне мира машин, поэтому разберём задачу про строки и кости.

Определение 8 (Соответствие Поста (*Post correspondence*))

Дан конечный набор костей $d_1, \ldots, d_k$, у кости d_i наверху строка t_i, внизу — строка b_i.

Набор имеет **совпадение**, если непустая последовательность костей даёт равные верхнюю и нижнюю цепочки.

Перебор последовательностей годится только на «да» — совпадение предъявляется, а его отсутствие нет.

Пример (Совпадение и его отсутствие)

У d_1 верх 1 и низ 111, у d_2 верх 11111 и низ 1. Последовательность $d_1 d_1 d_2$ складывает сверху $1 + 1 + 11111$, снизу $111 + 111 + 1$, и цепочки сходятся.

Кость с верхом 1 и низом 11 совпадений не имеет — сверху k единиц, снизу $2k$.

Неразрешимость соответствий Поста

Машины сводятся даже к костям домино.

Теорема 10 (Неразрешимость PCP)

Язык PCP неразрешим.

Доказательство

Докажем сведением от HALT.

По паре $(\langle M \rangle, w)$ строим набор костей. Стартовая кость задаёт начальную конфигурацию $q_0 w$, кости-шаги переносят конфигурацию в следующую, а финальная кость закрывает принимающую.

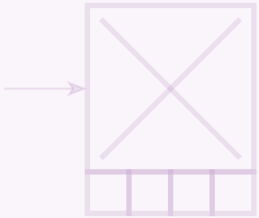
Совпадение вынуждено, ведь первой идёт только стартовая кость, а дальше верхнюю цепочку достраивает единственная подходящая кость. Поэтому совпадение есть ровно при остановке M на w, и решатель PCP решал бы HALT.

Цепочка неразрешимостей

Каждая строка — сведение от уже известной неразрешимой задачи.

Задача	Вопрос	Откуда неразрешимость
HALT	останавливается ли M на w?	диагональ
$\mathrm{HALT}_\varepsilon$	есть ли остановка на пустом входе?	$\mathrm{HALT} \leq_m \mathrm{HALT}_\varepsilon$
E_{TM}	пуст ли язык машины?	$\mathrm{HALT} \leq_m E_{\mathrm{TM}}$
TOT	тотальна ли машина?	$\mathrm{HALT} \leq_m \mathrm{TOT}$
$\mathrm{EQ}_{\mathrm{TM}}$	совпадают ли два языка?	$E_{\mathrm{TM}} \leq_m \mathrm{EQ}_{\mathrm{TM}}$
PCP	есть ли совпадение?	$\mathrm{HALT} \leq_m \mathrm{PCP}$

Классы расходятся. Языки HALT, $\mathrm{HALT}_\varepsilon$ и PCP распознаваемы и не разрешимы, E_{TM} ко-распознаваем, а TOT и $\mathrm{EQ}_{\mathrm{TM}}$ не попадают ни в один из двух классов.



Теорема Райса

> *Остерегайтесь ошибок в приведённом выше коде: я лишь доказал его корректность, но не исполнил его.*
>
> **— Дональд Кнут**

Свойство языка или свойство машины?

Пустота, эквивалентность, универсальность — каждое свойство потребовало своего сведения.

Определение 9 (Экстенсиональность и нетривиальность)

Свойство P описаний машин **экстенсионально**, если оно зависит только от языка.

$$L(M_1) = L(M_2) \quad \Rightarrow \quad (\langle M_1 \rangle \in P \leftrightarrow \langle M_2 \rangle \in P).$$

Свойство **нетривиально**, если существуют машины и с P, и без P.

Пример (Экстенсионально или нет)

«Язык пуст» — экстенсионально и нетривиально.

Не экстенсиональны «не более 100 состояний» и «принимает ли M слово 01 за сто шагов». Оба читают устройство машины, а не множество её слов.

Формулировка теоремы Райса

Отдельные сведения для каждого свойства дали бы бесконечную работу, а Райс обрывает её одним утверждением.

Теорема 11 (Райс, 1953)

Всякое нетривиальное экстенсиональное свойство распознаваемых языков неразрешимо.

Иначе говоря, для каждого нетривиального P язык $\{\langle M\rangle \mid \langle M\rangle \in P\}$ неразрешим.

Пример (Что закрывается одним ударом?)

Пуст ли язык? Конечен ли он? Регулярен ли он? Содержит ли он слово 01? Совпадает ли он с Σ^*?

Каждое свойство экстенсионально и нетривиально — каждое неразрешимо.

Разрешимы лишь тривиальные свойства, верные для всех языков или ни для одного.

Доказательство Райса

Схема повторяет сведение к пустоте языка.

Доказательство

Докажем сведением HALT к P.

Без ограничения общности машина с пустым языком не лежит в P. Иначе берём дополнение свойства — они разрешимы вместе.

Нетривиальность даёт машину A с $\langle A \rangle \in P$.

По паре $(\langle M \rangle, w)$ строим M'. На входе x она игнорирует x, симулирует M на w, а после остановки отдаёт x машине A.

Если M остановилась на w, то $L(M') = L(A)$ и $\langle M' \rangle \in P$. Если M зациклилась, то $L(M') = \varnothing$ и $\langle M' \rangle \notin P$.

Разрешитель P на $\langle M' \rangle$ отвечает на вопрос остановки — противоречие.

Сведение на конкретном свойстве

Проверим обе посылки теоремы на одном свойстве и проведём сведение целиком.

Пример (Свойство «пустая строка в языке»)

Свойство P состоит в том, что $\varepsilon \in L(M)$.

Экстенсиональность очевидна. Язык — в точности множество принимаемых слов, поэтому принадлежность ε зависит лишь от языка.

Свойство нетривиально. Машина, принимающая всё, им обладает, а машина M_{loop} — нет.

По паре $(\langle M \rangle, w)$ строим M' из доказательства. Машина M' симулирует M на w и при остановке принимает.

Если M остановилась, то M' принимает каждое слово, включая ε, и свойство есть. Если M зациклилась, то M' не принимает ничего, и свойства нет.

Где кончается теорема Райса?

Три свойства вне досягаемости

Свойство «у машины не более 100 состояний» не экстенсионально и к тому же разрешимо. Число состояний читается из кодировки.

Свойство «останавливается ли M на пустом входе за сто шагов» не экстенсионально и разрешимо. Достаточно симуляции ста шагов.

Свойство «останавливается ли M на пустом входе» не экстенсионально, и теорема Райса его не закрывает. Неразрешимость доказывается отдельным сведением.

Граница Райса — граница экстенсиональности, а не граница неразрешимости.

За границей живут и разрешимые свойства (число состояний), и неразрешимые (поведение на входе).

Инженерные следствия

Пример (Три инструмента и их пределы)

Верификатор завершения отвечает на вопрос тотальности — неразрешимость тотальности доказана сведением.

Проверка рефакторинга отвечает на вопрос эквивалентности старого и нового кода — неразрешимость эквивалентности доказана сведением.

Идеальный антивирус по программе V, выполняющей вредоносный код при остановке симулируемой M, решал бы HALT.

Универсального верификатора и идеального антивируса не существует, потому что оба решали бы нетривиальные свойства поведения программ.

Инструменты выживают третьим ответом — «да», «нет» или «не знаю».

За пределами машин Тьюринга

> *Любая достаточно развитая технология неотличима от магии.*
>
> ***— Артур Кларк***

Оракулы

Насколько неразрешимость «трудна»? Задачи сравнивают через гипотетических помощников.

Определение 10 (МТ с оракулом)

МТ получает дополнительную ленту-оракул. Записав строку и перейдя в состояние запроса, машина за один шаг узнаёт, принадлежит ли строка языку A.

Оракул — чёрный ящик, решающий A, то есть подпрограмма, отвечающая мгновенно.

Пример (Оракул для «HALT»)

Даже он не решает тотальность. Вопрос «для всех ли x машина останавливается?» требует бесконечной серии запросов — по одному на каждый вход.

Лестница иерархии

Неразрешимость не одинакова, и логическая сложность определения делит её на уровни.

Уровень	Задача	Кванторная форма
Δ_1	разрешимые задачи	ответ без неограниченных кванторов
Σ_1	проблема остановки	$\exists t.$ остановится за t шагов
Π_1	дополнение остановки	$\forall t.$ не остановится за t шагов
Σ_2	конечность языка	$\exists n \forall w.$ слово длиннее n не принимается
Π_2	тотальность	$\forall x \exists t.$ остановится на каждом x за t шагов
Σ_3	коконечность языка	$\exists n \forall w \exists t.$ слово длиннее n принимается за t шагов

Кванторная глубина — мост к логике первого порядка.

Оракул для HALT решает Σ_1-вопросы, а Π_2-тотальность требует оракула уровнем выше — иерархия не кончается.

Функция занятого бобра

Невычислимое встречается и без оракулов, максимум по конечному набору программ.

Определение 11 (Занятый бобёр (*busy beaver*))

$\Sigma(n)$ — максимум единиц, которые запишет останавливающаяся машина с n состояниями и алфавитом $\{0, 1\}$ на пустой ленте, а $S(n)$ — максимум её шагов.

Σ мажорирует любую вычислимую функцию, то есть $\Sigma(n) > f(n)$ при достаточно больших n для всякой вычислимой f.

Машина, вычисляющая f, тратит c состояний, ещё $\log n$ уходит на запись числа n, поэтому при больших n выходит $\Sigma(n) \geq \Sigma(\log n + c) \geq f(n) + 1$.

Известны $\Sigma(1) = 1$, $\Sigma(2) = 4$, $\Sigma(3) = 6$, $\Sigma(4) = 13$ и $\Sigma(5) = 4098$, а для $\Sigma(6)$ нижняя оценка выше башни из пятнадцати экспонент.

Гипервычисления

Гипервычисления — спекуляции о машинах, выходящих за тезис. Сюда относят бесконечное ускорение, оракулы в физике и точные действительные числа за один шаг.

Тезис Чёрча–Тьюринга — не теорема, и физика не обязана подчиняться определению.

Но за девяносто лет ни одна реализация не вышла за класс тьюринг-вычислимого. Гипотез хватает, работающих машин — нет.

Машина Зенона
n-й шаг занимает 2^{-n} секунды, поэтому бесконечный прогон укладывается в одну секунду. Такая машина решала бы остановку, ведь к моменту $t = 1$ весь прогон уже позади. Физической реализации у неё нет, поэтому тезис она не опровергает.

Десятая проблема Гильберта

Неразрешимость живёт не только в вопросах о программах — целочисленные уравнения упираются в ту же границу.

Определение 12 (Диофантово уравнение)

Уравнение $p(x_1, ..., x_n) = 0$, где p — многочлен с целыми коэффициентами, называется **диофантовым**.

Решением уравнения служит набор целых значений, обращающий многочлен в ноль.

Пример (Разное поведение уравнений)

$x^2 + y^2 = z^2$ разрешимо. Свидетель — $3^2 + 4^2 = 5^2$.

$x^2 + y^2 = 3$ неразрешимо. Квадрат целого числа даёт остаток 0 или 1 по модулю 4, и сумма двух квадратов не даёт 3.

Гильберт в 1900 году просил алгоритм, разделяющий уравнения этих двух сортов.

Неразрешимость десятой проблемы

Разделить разрешимые и неразрешимые уравнения тоже не выходит.

Теорема 12 (Десятая проблема Гильберта (MRDP, 1970))

Не существует алгоритма, определяющего по диофантову уравнению, есть ли у него решение в целых числах.

Доказательство

Докажем сведением от проблемы остановки.

По теореме MRDP всякое перечислимое множество диофантово. Множество пар (m, w) с остановкой машины m на w перечислимо, поэтому найдётся многочлен p, у которого $p(m, w, y_1, ..., y_k) = 0$ разрешимо ровно при остановке m на w.

Решатель диофантовых уравнений отвечал бы тогда на вопрос остановки для любой пары, значит, такого алгоритма нет.

Что с этим делать инженеру?

Неразрешимость не запрещает практику, а задаёт её форму. Инструменты работают на фрагментах и приближениях.

Если задача неразрешима на машине Тьюринга, её не решит ни один язык программирования.

Остаётся искать разрешимые фрагменты, ограничивать класс входов и строить приближения.

Граница вычислимого не зависит от модели, и динамические языки, нейросети и квантовые вычисления её не сдвигают.

Разрешимость и сведения

Три класса дают три уровня гарантий, а именно полный ответ, гарантию для своих слов и гарантию для чужих.

Диагональ, сведения и теорема Райса провели границу.

- Разрешитель отвечает на каждом входе, распознаватель — только для своих слов, ко-распознаватель — только для чужих.
- Перечислитель равносилен распознавателю, разрешимость — распознаваемость с двух сторон, а дополнение HALT показывает, что второй стороны может не быть.
- Диагональ запрещает решатель остановки, а сведения разносят неразрешимость на пустоту, эквивалентность и домино Поста.
- Теорема Райса закрывает нетривиальные свойства языка — отсюда пределы анализаторов.
- Оракулы и десятая проблема Гильберта переносят границу вне программ — в целочисленные уравнения.

Вопросы по разрешимости

о разрешимости и теореме Райса

1. Чем разрешитель отличается от распознавателя и ко-распознавателя? Какой из классов замкнут относительно дополнения? Почему два других класса этим свойством не обладают?
2. Восстановите оба перехода в равносильности перечислителя и распознавателя.
3. Воспроизведите диагональ. Как машина D использует H и где в рассуждении появляется собственный код?
4. Сведите пустоту языка к эквивалентности программ. Какую машину ставит рядом сводящая функция?
5. Опишите набор костей Поста для пары $(\langle M \rangle, w)$ и то, что читается в совпадении.
6. Применима ли теорема Райса к свойству «у машины ровно три состояния»? Почему свойство «останавливается ли M на пустом входе» вне её досягаемости?

Что почитать?

Чтение из главы m26 закрывает теорему Райса и разделы за пределами машин Тьюринга.

Задача	Где читать
Понять классы	m26, раздел «Разрешимость»
Доказать неразрешимость	m26, раздел «Проблема остановки»
Применить Райса	m26, раздел «Теорема Райса»
Выйти за Тьюринга	m26, раздел «За пределами машин Тьюринга»